

Information Retrieval: Embeddings Systems

Kajsa Vesterberg

May 9, 2026

1 Links

As requested, the links to the repo, dataset, and demo are as can be seen below, as well as a model artifact for the best performing model:

Repo: <https://github.com/kvest42/Swedish-CEFR-Classifier/tree/master>

Dataset: <https://huggingface.co/datasets/kvest/swedish-cefr-text-complexity>

Demo: <https://huggingface.co/spaces/kvest/swedish-cefr-text-classifier>

Model Artifact: <https://huggingface.co/kvest/swedish-cefr-linear-svm>

2 Reflection

For this project, I initially explored several different ideas before deciding on the final direction. My first intention was to create something focused on Swedish dialects, as I find regional language variation very interesting. However, after some reflection and preliminary research, I realized that this would be difficult to implement effectively within the scope of the project. Most dialectal differences are primarily phonetic rather than textual, which would require speech-based data and more advanced processing methods than I had originally anticipated. Because of this, I decided to move toward a text-based classification task instead.

The final idea became a system for categorizing Swedish texts according to their reading difficulty. At first, I considered a simpler categorization into “easy,” “medium,” and “hard” to read. Later, I refined the project further by connecting the classifications to the CEFR scale (Common European Framework of Reference for Languages). I found this especially interesting because CEFR levels are widely used in language learning and provide a more structured and meaningful framework for evaluating text complexity. The project therefore became both a text classification task and an exploration of how machine learning can be applied to language accessibility. I then generated a dataset which contained a hundred sentences of each level of CEFR, and manually went through and double-checked that the sentences seemed reasonable.

One aspect of the project that I found particularly fascinating was working with the AI-assisted programming tool Codex for the first time. Before this project, I had not fully understood how advanced these systems had become. In many cases, the AI was able to generate code significantly faster and more efficiently than I could have done on my own. This was especially noticeable in areas such as error handling, structuring the application, and integrating the project with external platforms like Hugging Face. I was surprised by how seamlessly it could generate functioning solutions for tasks that I would

otherwise have spent many hours implementing manually. Admittedly, this was a bit discouraging, but the industry is rapidly adapting toward favoring AI, and thus we need to as well.

At the same time, the experience also highlighted some of the limitations of AI-assisted coding. Although the generated code was often highly effective, the system occasionally produced errors or overly complicated solutions. This was interesting to observe, since it demonstrated that AI tools are powerful but still require human oversight and critical evaluation. In several cases, I found it necessary to simplify parts of the implementation manually or adjust the generated code to better suit the goals of the project. Overall, the experience gave me a greater understanding of both the strengths and limitations of modern AI systems, as well as their potential role as collaborative tools in software development and natural language processing projects.

This reflection was written by me but refined using AI.